Imam Abdulrahman Bin Faisal University
College of Computer Science and Information Technology

ARTI 404 — Natural Language Processing

Lab 10 — Introduction to Machine Translation

Student Name	Ahmed Albuainain
Student ID	2240006128
Section	7MA1

Introduction to Machine Translation (MT):

Machine Translation is a subfield of AI and linguistics that develops systems capable of translating text from one language to another. The primary goal of machine translation is to enable effective communication between people who speak different languages by automating the process of language translation.

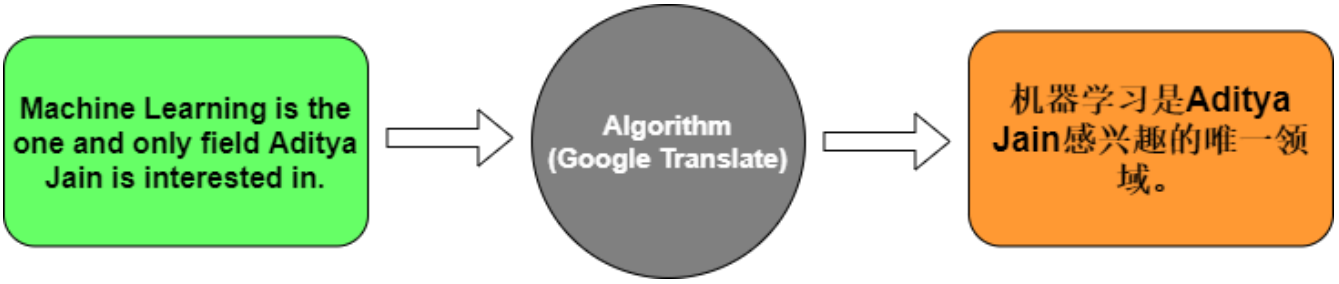
Setup

Run this cell first. It installs TensorFlow and kagglehub, which are used in this lab.

```
# Install the libraries used in this lab (safe to re-run)
import sys
!{sys.executable} -m pip install -q tensorflow kagglehub

import tensorflow as tf
print("TensorFlow version:", tf.__version__)
print("Setup complete.")
```

```
TensorFlow version: 2.21.0
Setup complete.
```



Challenges in Machine Translation:
- Ambiguity

Ex. I went to the bank.
- Idiomatic Expressions

Ex. It's raining cats and dogs.
- Low-Resource Languages

Some languages have very limited data.

Applications of Machine Translation:
- Global Business and E-commerce
-Cross-Cultural Communication
- Multilingual Content Creation
- Language Learning

Neural Machine Translation:

Neural Machine Translation is a data-driven approach where a single large neural network learns to represent both the source language and the target language, and to directly map

between them.
Instead of using manually defined rules or phrase tables, it learns translation patterns automatically from large datasets of bilingual text.

Key Components of Neural Machine Translation:



```
#!pip install tensorflow
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
import numpy as np

# Example bilingual dataset
english_sentences = ["Hello.", "How are you?", "This is a machine translation example."]
french_sentences = ["Bonjour.", "Comment ça va?", "Ceci est un exemple de traduction
automatique."]

# Tokenize the english sentences
tokenizer_eng = Tokenizer()
tokenizer_eng.fit_on_texts(english_sentences) # Tokenize words and assign an integer ID
to each unique word
eng_sequences = tokenizer_eng.texts_to_sequences(english_sentences) # Returns a list of
tokens replaced by their index. Note: words index will start from 1 because 0 is
reserved by default for padding (not assigned to any word).
print(f"eng_sequences = {eng_sequences}")
eng_word_index = tokenizer_eng.word_index # Returns the dictionary created by the
tokenizer dict {word: index}.
print(f"eng_word_index = {eng_word_index}")
```

```
eng_sequences = [[1], [2, 3, 4], [5, 6, 7, 8, 9, 10]]
eng_word_index = {'hello': 1, 'how': 2, 'are': 3, 'you': 4, 'this': 5, 'is': 6, 'a': 7,
'machine': 8, 'translation': 9, 'example': 10}
```

```
# Tokenize the french sentences
tokenizer_frn = Tokenizer()
tokenizer_frn.fit_on_texts(french_sentences)
frn_sequences = tokenizer_frn.texts_to_sequences(french_sentences)

print(f"frn_sequences = {frn_sequences}")
frn_word_index = tokenizer_frn.word_index
print(f"frn_word_index = {frn_word_index}")
```

```
frn_sequences = [[1], [2, 3, 4], [5, 6, 7, 8, 9, 10, 11]]
frn_word_index = {'bonjour': 1, 'comment': 2, 'ça': 3, 'va': 4, 'ceci': 5, 'est': 6, 'un': 7,
'exemple': 8, 'de': 9, 'traduction': 10, 'automatique': 11}
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Padding sequences
eng_padded = pad_sequences(eng_sequences) # makes all sequences the same length by
padding with zeros at the beginning (default)
print(f"eng_padded = {eng_padded}")
```

```
eng_padded = [[ 0  0  0  0  0  1]
 [ 0  0  0  2  3  4]
 [ 5  6  7  8  9 10]]
```

```
frn_padded = pad_sequences(frn_sequences)
frn_padded
```

```
array([[ 0,  0,  0,  0,  0,  0,  1],
       [ 0,  0,  0,  0,  2,  3,  4],
       [ 5,  6,  7,  8,  9, 10, 11]], dtype=int32)
```

LSTM Model



```python
from sklearn.model_selection import train_test_split

# Train-test split
eng_train, eng_test, frn_train, frn_test = train_test_split(eng_padded, frn_padded,
test_size=0.2, random_state=42)

# Define the model
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(eng_word_index) + 1, output_dim=64),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.RepeatVector(frn_padded.shape[1]), #length of french sentence
    tf.keras.layers.LSTM(64, return_sequences=True),
    tf.keras.layers.TimeDistributed(tf.keras.layers.Dense(len(frn_word_index) + 1,
activation='softmax')) #convert to probabilities for each frn word
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=
['accuracy'])

# Train the model
model.fit(eng_train, np.expand_dims(frn_train, -1), epochs=50, batch_size=1,
validation_split=0.2)

# Evaluate the model
loss, accuracy = model.evaluate(eng_test, np.expand_dims(frn_test, -1))
print(f'Test Accuracy: {accuracy}')
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >=
2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
TensorFlow-DirectML plugin.
Epoch 1/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 6s 6s/step - accuracy: 0.0000e+00 - loss: 2.4827 - val_accuracy:
0.0000e+00 - val_loss: 2.4878
Epoch 2/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 170ms/step - accuracy: 0.5714 - loss: 2.4733 - val_accuracy:
0.0000e+00 - val_loss: 2.4901
Epoch 3/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 164ms/step - accuracy: 0.5714 - loss: 2.4635 - val_accuracy:
0.0000e+00 - val_loss: 2.4926
Epoch 4/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 145ms/step - accuracy: 0.5714 - loss: 2.4529 - val_accuracy:
0.0000e+00 - val_loss: 2.4954
Epoch 5/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 151ms/step - accuracy: 0.5714 - loss: 2.4410 - val_accuracy:
0.0000e+00 - val_loss: 2.4985
Epoch 6/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 198ms/step - accuracy: 0.5714 - loss: 2.4273 - val_accuracy:
0.0000e+00 - val_loss: 2.5021
Epoch 7/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 167ms/step - accuracy: 0.5714 - loss: 2.4114 - val_accuracy:
0.0000e+00 - val_loss: 2.5063
Epoch 8/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 251ms/step - accuracy: 0.5714 - loss: 2.3927 - val_accuracy:
0.0000e+00 - val_loss: 2.5112
Epoch 9/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 134ms/step - accuracy: 0.5714 - loss: 2.3705 - val_accuracy:
0.0000e+00 - val_loss: 2.5169
Epoch 10/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 147ms/step - accuracy: 0.5714 - loss: 2.3441 - val_accuracy:
0.0000e+00 - val_loss: 2.5237
Epoch 11/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 139ms/step - accuracy: 0.5714 - loss: 2.3125 - val_accuracy:
0.0000e+00 - val_loss: 2.5318
Epoch 12/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 207ms/step - accuracy: 0.5714 - loss: 2.2747 - val_accuracy:
0.0000e+00 - val_loss: 2.5415
Epoch 13/50
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 158ms/step - accuracy: 0.5714 - loss: 2.2297 - val_accuracy:
0.0000e+00 - val_loss: 2.5532
Epoch 14/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 147ms/step - accuracy: 0.5714 - loss: 2.1763 - val_accuracy:
0.0000e+00 - val_loss: 2.5676
Epoch 15/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 160ms/step - accuracy: 0.5714 - loss: 2.1138 - val_accuracy:
0.0000e+00 - val_loss: 2.5855
Epoch 16/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 169ms/step - accuracy: 0.5714 - loss: 2.0423 - val_accuracy:
0.0000e+00 - val_loss: 2.6080
Epoch 17/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 143ms/step - accuracy: 0.5714 - loss: 1.9637 - val_accuracy:
0.0000e+00 - val_loss: 2.6367
Epoch 18/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 159ms/step - accuracy: 0.5714 - loss: 1.8823 - val_accuracy:
0.0000e+00 - val_loss: 2.6736
Epoch 19/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 134ms/step - accuracy: 0.5714 - loss: 1.8047 - val_accuracy:
0.0000e+00 - val_loss: 2.7205
Epoch 20/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 165ms/step - accuracy: 0.5714 - loss: 1.7364 - val_accuracy:
0.0000e+00 - val_loss: 2.7788
Epoch 21/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step - accuracy: 0.5714 - loss: 1.6778 - val_accuracy:
0.0000e+00 - val_loss: 2.8485
Epoch 22/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 141ms/step - accuracy: 0.5714 - loss: 1.6242 - val_accuracy:
0.0000e+00 - val_loss: 2.9292
Epoch 23/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 145ms/step - accuracy: 0.5714 - loss: 1.5705 - val_accuracy:
0.0000e+00 - val_loss: 3.0203
Epoch 24/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 144ms/step - accuracy: 0.5714 - loss: 1.5141 - val_accuracy:
0.0000e+00 - val_loss: 3.1205
Epoch 25/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step - accuracy: 0.5714 - loss: 1.4548 - val_accuracy:
0.0000e+00 - val_loss: 3.2280
Epoch 26/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 132ms/step - accuracy: 0.5714 - loss: 1.3938 - val_accuracy:
0.0000e+00 - val_loss: 3.3405
Epoch 27/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 132ms/step - accuracy: 0.5714 - loss: 1.3324 - val_accuracy:
0.0000e+00 - val_loss: 3.4548
Epoch 28/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 193ms/step - accuracy: 0.5714 - loss: 1.2714 - val_accuracy:
0.0000e+00 - val_loss: 3.5680
Epoch 29/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step - accuracy: 0.5714 - loss: 1.2110 - val_accuracy:
0.0000e+00 - val_loss: 3.6771
Epoch 30/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 149ms/step - accuracy: 0.5714 - loss: 1.1518 - val_accuracy:
0.0000e+00 - val_loss: 3.7801
Epoch 31/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 149ms/step - accuracy: 0.5714 - loss: 1.0948 - val_accuracy:
0.0000e+00 - val_loss: 3.8758
Epoch 32/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 145ms/step - accuracy: 0.5714 - loss: 1.0413 - val_accuracy:
0.0000e+00 - val_loss: 3.9643
Epoch 33/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 154ms/step - accuracy: 0.5714 - loss: 0.9928 - val_accuracy:
0.0000e+00 - val_loss: 4.0471
Epoch 34/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - accuracy: 0.5714 - loss: 0.9524 - val_accuracy:
0.0000e+00 - val_loss: 4.1266
Epoch 35/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 213ms/step - accuracy: 0.5714 - loss: 0.9227 - val_accuracy:
0.0000e+00 - val_loss: 4.2060
Epoch 36/50
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 144ms/step - accuracy: 0.5714 - loss: 0.9024 - val_accuracy:
0.0000e+00 - val_loss: 4.2881
Epoch 37/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 150ms/step - accuracy: 0.5714 - loss: 0.8856 - val_accuracy:
0.0000e+00 - val_loss: 4.3743
Epoch 38/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 140ms/step - accuracy: 0.5714 - loss: 0.8659 - val_accuracy:
0.0000e+00 - val_loss: 4.4642
Epoch 39/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 137ms/step - accuracy: 0.5714 - loss: 0.8413 - val_accuracy:
0.0000e+00 - val_loss: 4.5568
Epoch 40/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 141ms/step - accuracy: 0.7143 - loss: 0.8149 - val_accuracy:
0.0000e+00 - val_loss: 4.6508
Epoch 41/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step - accuracy: 0.7143 - loss: 0.7928 - val_accuracy:
0.0000e+00 - val_loss: 4.7443
Epoch 42/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 197ms/step - accuracy: 0.7143 - loss: 0.7760 - val_accuracy:
0.0000e+00 - val_loss: 4.8354
Epoch 43/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 135ms/step - accuracy: 0.7143 - loss: 0.7581 - val_accuracy:
0.0000e+00 - val_loss: 4.9224
Epoch 44/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 136ms/step - accuracy: 0.7143 - loss: 0.7387 - val_accuracy:
0.0000e+00 - val_loss: 5.0042
Epoch 45/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 152ms/step - accuracy: 0.7143 - loss: 0.7219 - val_accuracy:
0.0000e+00 - val_loss: 5.0790
Epoch 46/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 157ms/step - accuracy: 0.7143 - loss: 0.7080 - val_accuracy:
0.0000e+00 - val_loss: 5.1465
Epoch 47/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 152ms/step - accuracy: 0.7143 - loss: 0.6957 - val_accuracy:
0.0000e+00 - val_loss: 5.2087
Epoch 48/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 183ms/step - accuracy: 0.7143 - loss: 0.6834 - val_accuracy:
0.0000e+00 - val_loss: 5.2683
Epoch 49/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - accuracy: 0.7143 - loss: 0.6682 - val_accuracy:
0.0000e+00 - val_loss: 5.3247
Epoch 50/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 139ms/step - accuracy: 0.7143 - loss: 0.6526 - val_accuracy:
0.0000e+00 - val_loss: 5.3756
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 882ms/step - accuracy: 0.7143 - loss: 1.3758
Test Accuracy: 0.7142857313156128
```

```python
# Inference
def translate_sentence(sentence, model, eng_tokenizer, frn_tokenizer):
    sequence = eng_tokenizer.texts_to_sequences([sentence])
    padded_sequence = pad_sequences(sequence, maxlen=eng_padded.shape[1])
    predicted_sequence = model.predict(padded_sequence) #outputs probabilities for each
timestep (french padded sentence) and each unique word in the target vocabulary.
    translated_sequence = np.argmax(predicted_sequence, axis=-1) #Returns the index
(word ID) with the largest value at each timestep.
    translated_sentence = frn_tokenizer.sequences_to_texts(translated_sequence)
    return translated_sentence[0]

# Example Inference
input_sentence = "How are you?"
translated_sentence = translate_sentence(input_sentence, model, tokenizer_eng,
tokenizer_frn)
print(f'Input: {input_sentence}\nTranslation: {translated_sentence}')
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 602ms/step
Input: How are you?
Translation: va va
```

Task 1: Understanding the Model Layers

Answer these simple questions in your own words:

1. What does the **Embedding** layer do to a word index?
2. What is the role of the **Encoder LSTM**?
3. What does the **Decoder LSTM** do?
4. Why do we use `softmax` in the final layer?

Answers

1. What does the Embedding layer do to a word index?

The tokenizer turns every word into an integer index, and these integers carry no meaning on their own — the index 7 is not "bigger" or "closer" to 8 than to 200. The Embedding layer replaces each index with a dense vector of a fixed size (64 in this model), and the values of that
vector are **learned during training**. Words that appear in similar contexts end up with similar
vectors, so the network receives a representation that actually carries meaning instead of an arbitrary number. It is also far more efficient than one-hot encoding, which would need one dimension per word in the vocabulary.

2. What is the role of the Encoder LSTM?

The encoder reads the English sentence one word at a time and updates its internal state at every
step. After the last word it has compressed the whole sentence into a single fixed-size vector (the final hidden state), usually called the **context vector**. This vector is the model's summary
of what the sentence means, and it is the only thing passed to the decoder. In the code this is the
`LSTM(64)` layer, which returns just the final state because `return_sequences` is not set.

3. What does the Decoder LSTM do?

The decoder takes that context vector and generates the French sentence from it.
`RepeatVector`
first copies the context vector once for every position of the output sentence, and then `LSTM(64, return_sequences=True)` produces one hidden state for each of those positions. In other
words, the encoder answers "what does this sentence mean?" and the decoder answers "how do I say
that meaning in French, word by word?".

4. Why do we use softmax in the final layer?

The last layer is a `Dense` layer with one unit for every word in the French vocabulary, and it is wrapped in `TimeDistributed` so that it is applied at every position of the output. Softmax turns
the raw scores of those units into a **probability distribution**: every value is between 0 and 1

and the values add up to 1 at each timestep. This lets the model say how likely each French word is
at that position, so the predicted word is simply the one with the highest probability (`np.argmax`). Softmax is also what the `sparse_categorical_crossentropy` loss expects, since that
loss compares a predicted probability distribution with the index of the correct word.

Task 2:

Use Case: Language Translation (English-French)

A global e-commerce company wants to expand its reach to French-speaking customers and provide a seamless shopping experience. The company needs a language translation solution to automatically translate product listings, customer reviews, and other relevant content from English to French.

link: https://www.kaggle.com/datasets/devicharith/language-translation-englishfrench

```
# Task 2: English-French translation for an e-commerce company
# 1. Load the dataset
import kagglehub, os, glob, re
import numpy as np
import pandas as pd

path = kagglehub.dataset_download("devicharith/language-translation-englishfrench")
print("Path to dataset files:", path)
print("Files:", os.listdir(path))

csv_file = glob.glob(os.path.join(path, "*.csv"))[0]
data = pd.read_csv(csv_file)

print("\nShape:", data.shape)
print("Columns:", list(data.columns))
data.head()
```

```
Downloading to C:\Users\ahmad\.cache\kagglehub\datasets\devicharith\language-translation-
englishfrench\2.archive...
```

```
Extracting files...
```

```
Path to dataset files: C:\Users\ahmad\.cache\kagglehub\datasets\devicharith\language-
translation-englishfrench\versions\2
Files: ['eng_-french.csv']

Shape: (175621, 2)
Columns: ['English words/sentences', 'French words/sentences']
```

	English words/sentences	French words/sentences
0	Hi.	Salut!
1	Run!	Cours !
2	Run!	Courez !
3	Who?	Qui ?
4	Wow!	Ça alors !

```python
# 2. Pre-processing
data.columns = ["english", "french"]
data = data.dropna().drop_duplicates().reset_index(drop=True)


def clean_text(text):
    text = str(text).lower().strip()
    # keep letters (including the French accented ones), apostrophes and spaces
    text = re.sub(r"[^a-z\u00e0-\u00ff' ]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text


data["english"] = data["english"].apply(clean_text)
data["french"] = data["french"].apply(clean_text)

data["eng_len"] = data["english"].str.split().str.len()
data["frn_len"] = data["french"].str.split().str.len()

# Keep only short sentence pairs and take a sample, so that the model
# can be trained in a reasonable time on a normal laptop
subset = data[(data["eng_len"].between(1, 8)) & (data["frn_len"].between(1, 8))]
subset = subset.sample(n=10000, random_state=42).reset_index(drop=True)

print("Total pairs in the dataset :", len(data))
print("Pairs used in this task    :", len(subset))
print("\nExamples:")
subset[["english", "french"]].head(10)
```

```
Total pairs in the dataset : 175621
Pairs used in this task    : 10000

Examples:
```

	english	french
0	i have an invitation	j'ai une invitation
1	you'll sleep better tonight	tu dormiras mieux cette nuit
2	she smells bad	elle pue
3	bear in mind what he said	gardez en mémoire ce qu il a dit
4	it's all over for us	c'est plié pour nous
5	let's move on to another topic	passons maintenant à un autre sujet
6	i don't need that kind of help	je n'ai pas besoin de ce genre d'aide
7	i know you left early	je sais que tu es parti tôt
8	do you want anything	veux tu quelque chose
9	i hope you'll soon get well	j'espère que vous irez bientôt mieux

```python
# 3. Tokenization and padding
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

MAX_WORDS = 5000

eng_tok = Tokenizer(num_words=MAX_WORDS, oov_token="<unk>")
eng_tok.fit_on_texts(subset["english"])
eng_seq = eng_tok.texts_to_sequences(subset["english"])

frn_tok = Tokenizer(num_words=MAX_WORDS, oov_token="<unk>")
frn_tok.fit_on_texts(subset["french"])
frn_seq = frn_tok.texts_to_sequences(subset["french"])

eng_maxlen = max(len(s) for s in eng_seq)
frn_maxlen = max(len(s) for s in frn_seq)

X = pad_sequences(eng_seq, maxlen=eng_maxlen, padding="post")
y = pad_sequences(frn_seq, maxlen=frn_maxlen, padding="post")

eng_vocab = min(MAX_WORDS, len(eng_tok.word_index) + 1)
frn_vocab = min(MAX_WORDS, len(frn_tok.word_index) + 1)

print("English vocabulary size :", eng_vocab)
print("French  vocabulary size :", frn_vocab)
print("English max length      :", eng_maxlen)
print("French  max length      :", frn_maxlen)
print("\nX shape:", X.shape, "  y shape:", y.shape)
```

```
English vocabulary size : 4203
French  vocabulary size : 5000
English max length      : 8
French  max length      : 8

X shape: (10000, 8)   y shape: (10000, 8)
```

```python
# 4. Train-test split and the model
from sklearn.model_selection import train_test_split
import tensorflow as tf

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

print("Training pairs:", len(X_train))
print("Testing pairs :", len(X_test))

model2 = tf.keras.Sequential([
    # the word index becomes a dense learned vector
    tf.keras.layers.Embedding(input_dim=eng_vocab, output_dim=128),

    # Encoder: compresses the English sentence into one context vector
    tf.keras.layers.LSTM(128),

    # repeat that context vector once for every position of the French sentence
    tf.keras.layers.RepeatVector(frn_maxlen),

    # Decoder: produces one hidden state per output position
    tf.keras.layers.LSTM(128, return_sequences=True),

    # one probability per French word, at every position
    tf.keras.layers.TimeDistributed(
        tf.keras.layers.Dense(frn_vocab, activation="softmax"))
])

model2.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])

model2.summary()
```

```
Training pairs: 8000
Testing pairs : 2000
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_2 (LSTM)	?	0 (unbuilt)
repeat_vector_1 (RepeatVector)	?	0 (unbuilt)
lstm_3 (LSTM)	?	0 (unbuilt)
time_distributed_1 (TimeDistributed)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
# 5. Train the model
history = model2.fit(X_train, np.expand_dims(y_train, -1),
                     epochs=10,
                     batch_size=64,
                     validation_split=0.1,
                     verbose=1)

# 6. Evaluate on the test set
loss, accuracy = model2.evaluate(X_test, np.expand_dims(y_test, -1), verbose=0)
print(f"\nTest Loss     : {loss:.4f}")
print(f"Test Accuracy : {accuracy:.4f}")
```

```
Epoch 1/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 19s 116ms/step - accuracy: 0.3090 - loss: 5.6053 - val_accuracy:
0.3013 - val_loss: 5.1370
Epoch 2/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 106ms/step - accuracy: 0.3117 - loss: 4.9883 - val_accuracy:
0.3013 - val_loss: 5.0969
Epoch 3/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 107ms/step - accuracy: 0.3117 - loss: 4.8588 - val_accuracy:
0.3013 - val_loss: 4.9543
Epoch 4/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 106ms/step - accuracy: 0.3304 - loss: 4.7094 - val_accuracy:
0.3248 - val_loss: 4.8467
Epoch 5/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 106ms/step - accuracy: 0.3345 - loss: 4.6163 - val_accuracy:
0.3248 - val_loss: 4.7988
Epoch 6/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 106ms/step - accuracy: 0.3391 - loss: 4.5672 - val_accuracy:
0.3311 - val_loss: 4.7845
Epoch 7/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 107ms/step - accuracy: 0.3400 - loss: 4.5302 - val_accuracy:
0.3311 - val_loss: 4.7616
Epoch 8/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 107ms/step - accuracy: 0.3413 - loss: 4.5042 - val_accuracy:
0.3317 - val_loss: 4.7417
Epoch 9/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 107ms/step - accuracy: 0.3439 - loss: 4.4419 - val_accuracy:
0.3358 - val_loss: 4.6445
Epoch 10/10
113/113 ━━━━━━━━━━━━━━━━━━━━ 12s 106ms/step - accuracy: 0.3492 - loss: 4.3580 - val_accuracy:
0.3417 - val_loss: 4.6130

Test Loss     : 4.5575
Test Accuracy : 0.3529
```

```python
# 7. Translate new English sentences into French
def translate_sentence(sentence, model, eng_tokenizer, frn_tokenizer, maxlen):
    cleaned = clean_text(sentence)
    sequence = eng_tokenizer.texts_to_sequences([cleaned])
    padded = pad_sequences(sequence, maxlen=maxlen, padding="post")
    predicted = model.predict(padded, verbose=0)
    ids = np.argmax(predicted, axis=-1)
    return frn_tokenizer.sequences_to_texts(ids)[0]


# Sentences of the kind an e-commerce site would need to translate
test_sentences = [
    "how are you",
    "i am happy",
    "the price is good",
    "thank you",
    "this is new",
]

print(f"{'ENGLISH':<28} FRENCH (model output)")
print("-" * 72)
for sentence in test_sentences:
    translation = translate_sentence(sentence, model2, eng_tok, frn_tok, eng_maxlen)
    print(f"{sentence:<28} {translation}")
```

```
ENGLISH                      FRENCH (model output)
------------------------------------------------------------------------
how are you                  je est <unk> <unk> <unk> <unk> <unk> <unk>
i am happy                   je est <unk> <unk> <unk> <unk> <unk> <unk>
the price is good            je est <unk> <unk> <unk> <unk> <unk> <unk>
thank you                    je est <unk> <unk> <unk> <unk> <unk> <unk>
this is new                  je est <unk> <unk> <unk> <unk> <unk> <unk>
```

```python
# 8. Compare the model output with the real translation on test examples
sample_idx = np.random.RandomState(0).choice(len(X_test), 5, replace=False)

print("Comparison on five pairs taken from the test set:\n")
for i in sample_idx:
    english_text = eng_tok.sequences_to_texts([X_test[i]])[0]
    reference = frn_tok.sequences_to_texts([y_test[i]])[0]
    prediction = frn_tok.sequences_to_texts(
        np.argmax(model2.predict(X_test[i:i + 1], verbose=0), axis=-1))[0]

    print(f"English   : {english_text}")
    print(f"Reference : {reference}")
    print(f"Model     : {prediction}")
    print("-" * 72)
```

```
Comparison on five pairs taken from the test set:

English   : she will make a good wife <unk> <unk>
Reference : elle fera une bonne épouse <unk> <unk> <unk>
Model     : je ne pas <unk> <unk> <unk> <unk> <unk>
------------------------------------------------------------------------
English   : strawberries are made into jam <unk> <unk> <unk>
Reference : on fait de la confiture avec les fraises
Model     : je ne pas <unk> <unk> <unk> <unk> <unk>
------------------------------------------------------------------------
English   : am i mistaken <unk> <unk> <unk> <unk> <unk>
Reference : est ce que je me trompe <unk> <unk>
Model     : je est <unk> <unk> <unk> <unk> <unk> <unk>
------------------------------------------------------------------------
English   : you're lucky to have a job <unk> <unk>
Reference : vous êtes chanceuse d'avoir un emploi <unk> <unk>
Model     : je ne pas pas <unk> <unk> <unk> <unk>
------------------------------------------------------------------------
English   : we were looking for tom <unk> <unk> <unk>
Reference : nous cherchions tom <unk> <unk> <unk> <unk> <unk>
Model     : je ne pas <unk> <unk> <unk> <unk> <unk>
------------------------------------------------------------------------
```

Summary of the Tasks

Task 1 — The model layers. The four answers are written above the code. In short: the **Embedding** layer replaces each word index with a dense vector that is learned during training, so that words used in similar contexts end up with similar vectors; the **encoder LSTM** reads the English sentence and compresses it into one fixed-size context vector; the **decoder LSTM** expands that vector (after `RepeatVector` copies it once per output position) into one hidden state per French word; and **softmax** turns the scores of the final `Dense` layer into a probability distribution over the French vocabulary at every position, which is what `sparse_categorical_crossentropy` expects and what `np.argmax` reads to pick the predicted word.

Task 2 — English–French translation.

Item	Value
Pairs in the full dataset	175,621
Pairs used (short sentences, random sample)	10,000
Training / testing split	8,000 / 2,000

English vocabulary	4,203
French vocabulary (capped at 5,000)	5,000
Maximum length (both languages)	8 tokens
Training	10 epochs, batch size 64, Adam
Test loss / accuracy	4.5575 / **0.3529**

Why the sample was reduced. The full dataset has 175,621 sentence pairs. Training a recurrent sequence-to-sequence model on all of them on a CPU would take several hours, so only short pairs (eight tokens or fewer) were kept and 10,000 of them were sampled at random. Each epoch then took about 12 seconds.

Discussion of the result. The model did not learn to translate: it produced almost the same output for every input ("*je est …*" or "*je ne pas …*"), which is a model that has fallen back on the most frequent French words instead of using the input sentence. Four separate things explain this, and together they are the real lesson of the lab.

1. The accuracy figure is misleading. Accuracy was already **0.3090 in the first epoch**, before the model had learned anything at all. Most positions in the target sequences are padding zeros, and predicting a padding token is trivial, so roughly a third of the score comes from padding rather than from translation. This is why the comparison cell matters more than the accuracy: it shows the actual words produced.

2. The model is undertrained. The training loss was still falling at the last epoch (5.6053 → 4.3580) and the validation loss with it (5.1370 → 4.6130). Nothing had converged; ten epochs simply stopped the run early.

3. The architecture has an information bottleneck. The whole English sentence is compressed into a single 128-dimensional context vector, and `RepeatVector` then feeds **the same vector** to every output position. The decoder therefore has no way of knowing which part of the source sentence belongs to which output word, and the safest strategy it can learn is to emit the most common French words at every position. Real neural machine translation systems solve exactly this problem with an **attention mechanism**, which lets the decoder look back at individual source words at each step, and modern systems use transformers for the same reason.

4. The vocabulary is large relative to the data. The output layer has to choose between 5,000 French words at every position, while the model only sees 8,000 training sentences. That is very little evidence per word, so rare words are never learned.

A note on the printed output. The `<unk>` tokens that appear at the end of the sentences in the comparison cell are the **padding positions**, not predictions: index 0 is the padding value, and when the sequences are converted back to text that index is rendered using the out-of-vocabulary token. They appear in the reference translations as well, which confirms that they come from padding and not from the model.

A note on `model.summary()`. The table reports the layers as "unbuilt" with 0 parameters because, in Keras 3, a `Sequential` model that is not given an input shape builds its weights only when it first receives data. The parameters are created at the start of `model.fit()`, which is why training then runs normally.